



Delegation

Dr Mamadou Bousso



DELEGATION

- Utilise la composition comme outil de réutilisation.
- Deux objets sont impliqués dans le traitement d'une requête:
 - Un objet **récepteur** qui délègue les opérations à son **délégué**;
 - L'objet **récepteur** possède un objet **délégué**;
 - Le **récepteur** passe sa propre référence à son **délégué**;

DELEGATION

Avantage: permet
facilement la
combinaison des
comportements à
l'exécution;



Inconvénients:

- Perte d'efficacité à l'exécution;
- Logiciel dynamique fortement paramétré est difficile à comprendre;



DELEGATION

- Patrons utilisant la délégation:
 - Le modèle **Etat**
 - Le modèle **Stratégie**
 - Le modèle **Visiteur**



DELEGATION

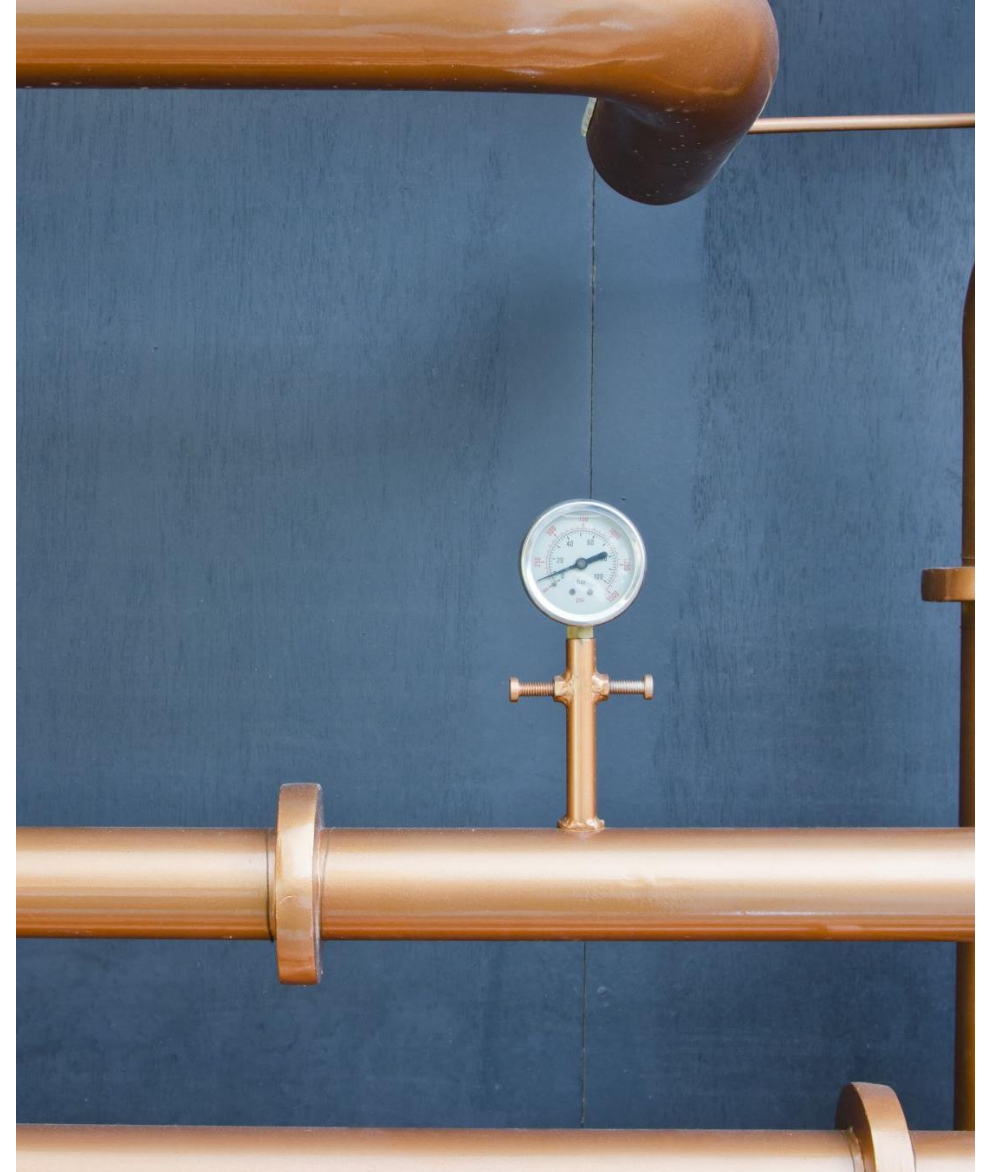
- Patron ETAT
 - Permet à un objet de modifier son comportement lorsque son état change

Patron ETAT

- Problème
 - L'état d'un objet est défini par la valeur de ses attributs;
 - Une modification des attributs amène un changement d'état;
 - **Pb: lorsque le changement d'état induit un changement de comportement de l'objet**
 - L'objet implante différentes variantes de son comportement;
 - Comportement choisi par des expressions conditionnelles

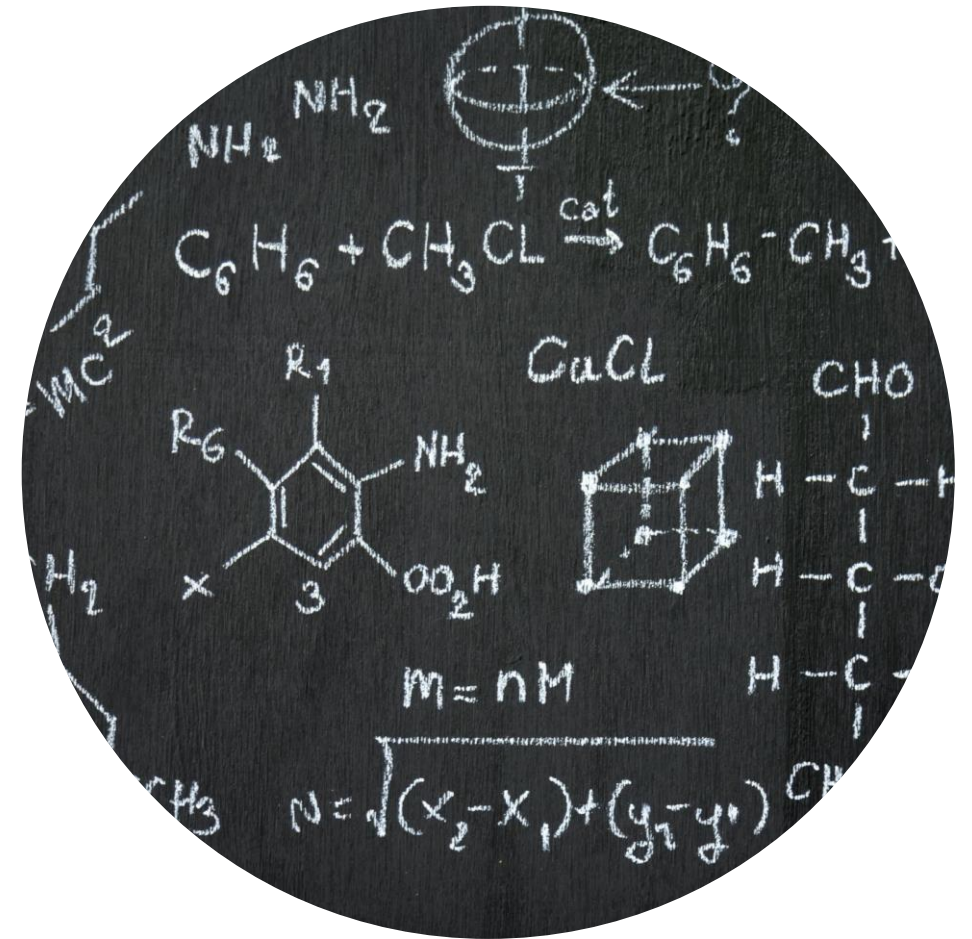
Patron ETAT

- **Exemple: système de gestion d'un barrage hydroélectrique**
 - Le barrage doit maintenir un certain niveau d'eau;
 - Pour contrôler le niveau d'eau, on peut **ouvrir les valves** ou **fermer les valves**;
 - lorsque les valves sont fermés, une accumulation naturelle s'effectue
- **3 états:**
 - Niveau bas -> Fermer les valves
 - Niveau normal -> Ne rien faire
 - Niveau élevé -> Ouvrir les valves



Patron ETAT

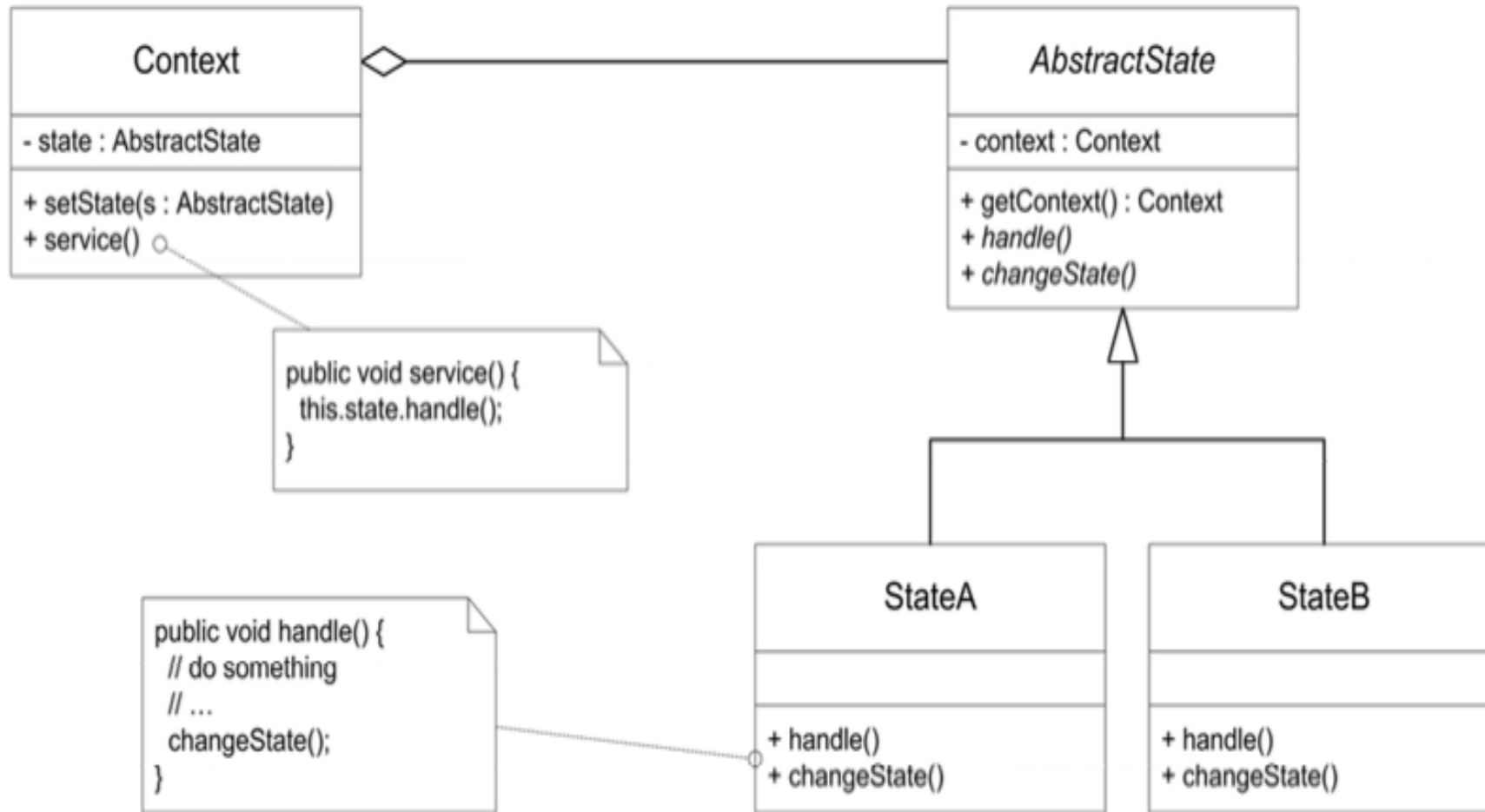
- **Exemple: système de gestion d'un barrage hydroélectrique**
 - Définir une classe **Barrage** avec des méthodes **ecouler(qte)** et **accumuler(qte)**
 - Ecrivez la méthode **ajusterNiveau()** qui permet d'ajuster le niveau d'eau en fonction des états et de la quantité d'eau. **La quantité d'eau maximale est de 200 et la quantité minimale est de 100;**



Patron ETAT

- **Solution:** encapsuler les différents comportements que peut prendre un objet en fonction de son état dans des classes distinctes (**Etat**) :
 - Les classes **Etat** implémentent toutes la même interface;
 - Une **classe parente abstraite** spécifie les méthodes de service qui **sont influencées** par l'état de l'objet;
 - La **classe concrete** implémente les services qui **ne sont pas influencés**

Patron ETAT



Patron ETAT

- Des sous-classes implantent les méthodes abstraites qui définissent le comportement de l'objet dans un état précis
 - Une sous-classe = Un état précis de l'objet
- Les requêtes sur les méthodes sensibles à l'état de l'objet sont redirigées vers la sous-classe qui représente l'état courant de l'objet
- Suite à une requête, l'état de l'objet est réévalué et une autre sous-classe est choisie



Patron ETAT

- **Conséquences :**

- Permet de regrouper les comportements en fonction du contexte dans lequel le comportement est applicable
- Facilite l'évolution et l'extension du comportement de l'objet;
- Élimine de nombreuses et complexes opérations conditionnelles
- Permet de structurer le comportement d'un objet

Patron ETAT

- **Conséquences :**

- Rend explicite les changements d'état
- Permet de partager des comportements entre plusieurs instances d'une classe
- La décentralisation de la logique des transitions d'état amène des dépendances entre les objets **Etat**
- Augmente le nombre d'objets du système

Patron VISITEUR

Définir de nouvelles opérations
sans modifier les classes des
éléments sur lesquelles les
opérations agissent

Patron VISITEUR

On veut définir une opération qui s'applique sur les éléments d'une **structure d'objets hétérogènes**;

On veut regrouper les différentes **versions de l'opération** (spécifiques à chaque type d'objet de la structure) dans une même classe;

- Une opération donnée s'implante d'une manière différente sur chacune des classes de la structure

On veut ajouter des opérations sans modifier les classes de la structure

Patron VISITEUR

Scénario: Système de fichier

On veut calculer la taille des éléments à partir d'un élément du système de fichier.

Fichier = taille du fichier

Repertoire = somme de la taille des éléments qu'il contient

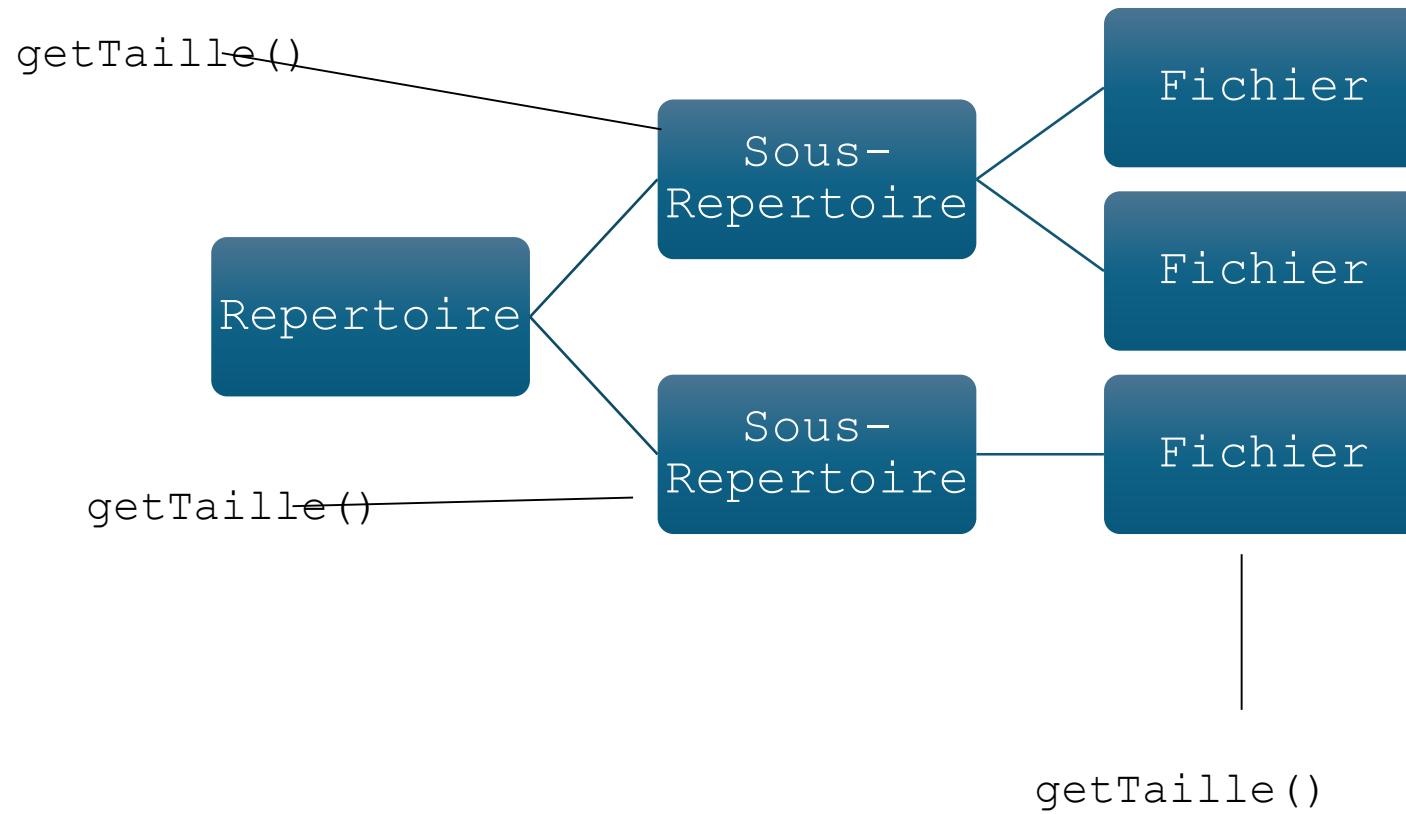
Que faut-il faire?

Mauvaise idée: Implanter une fonction qui calcule la taille pour chaque élément.

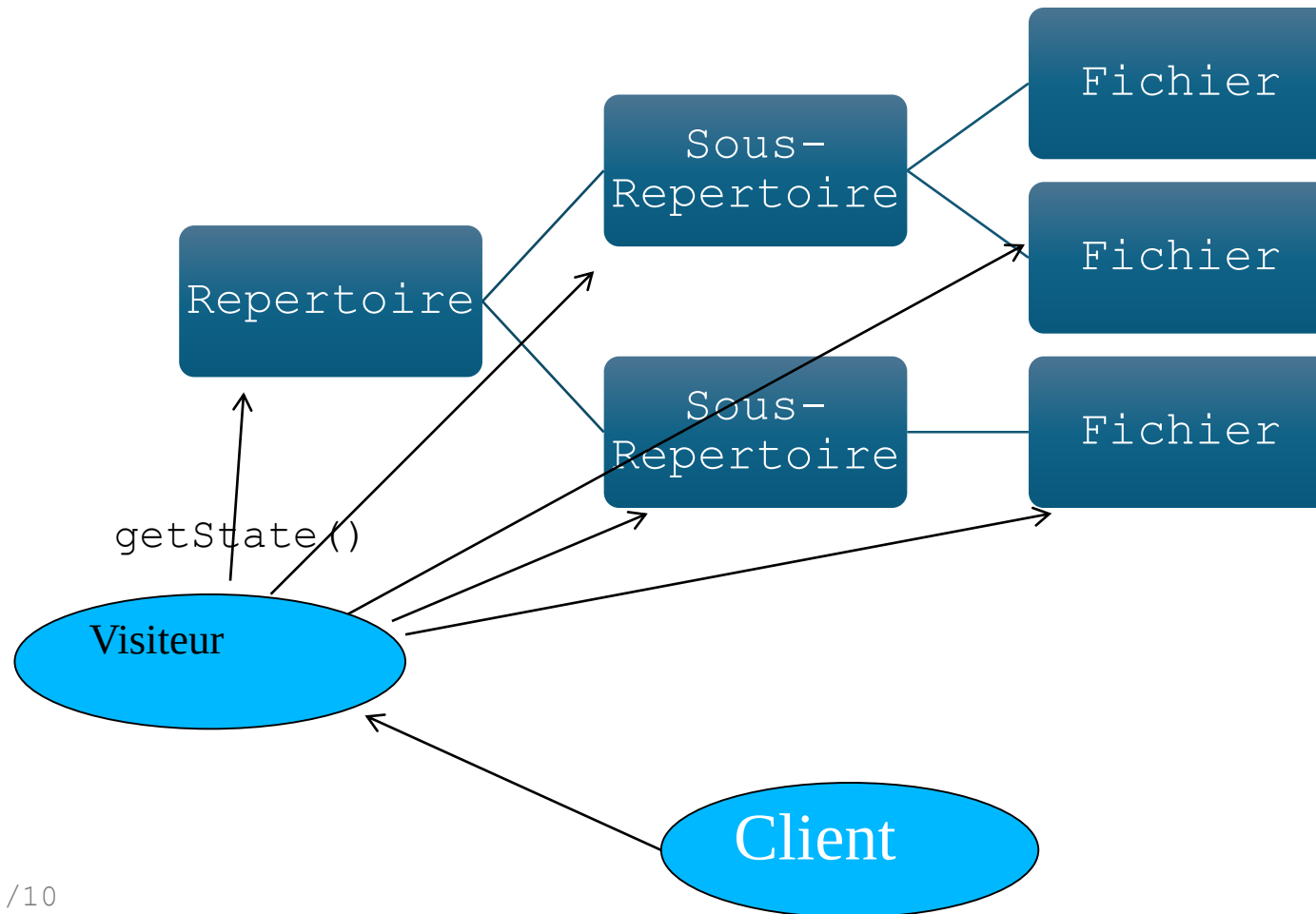
Bonne idée: Un visiteur qui parcourt la hiérarchie à partir d'un nœud et calcule la taille des éléments



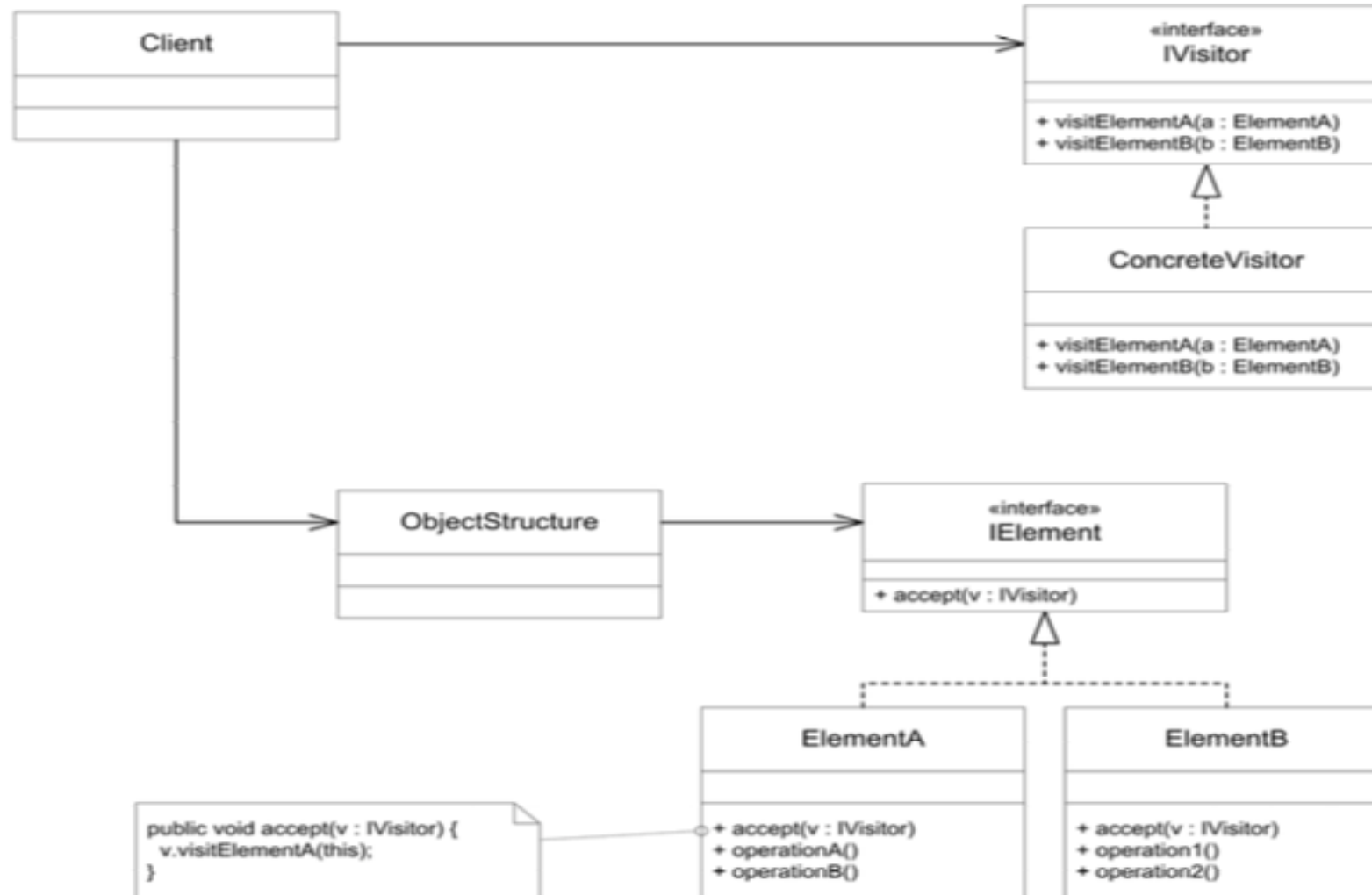
Patron VISITEUR



Patron VISITEUR



Patron VISITEUR

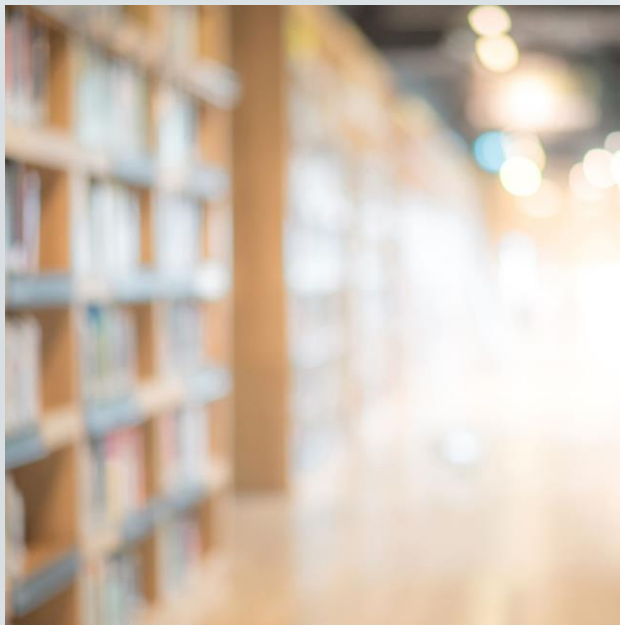


Patron VISITEUR

Solution

- Encapsuler l'opération dans une classe (Visiteur) ;
- Séparer les implantations des données

Définir une interface qui spécifie, pour chacune des classes, une méthode **visit(param : type)** qui prend en paramètre une référence sur un objet du type de la classe,



Patron VISITEUR

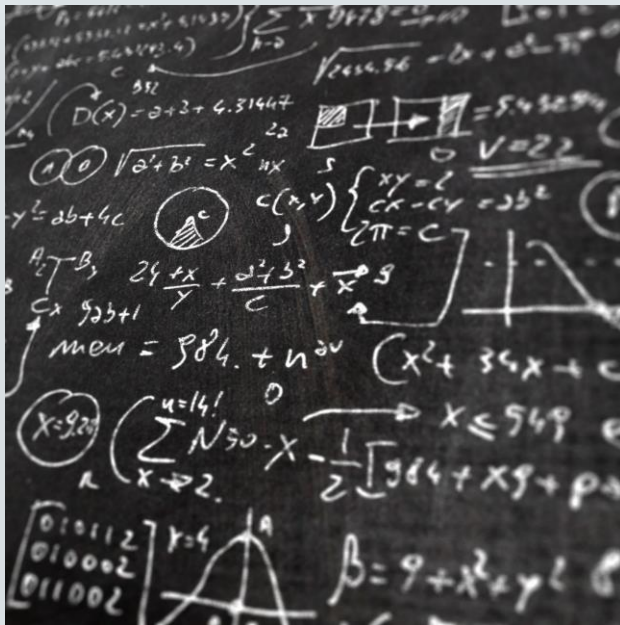


Solution

- Le visiteur implémente cet interface;
- Chaque méthode **visit()** implémente une version spécifique de l'opération représentée;
- Ces méthodes font appel au service des classes de la structure

Définir une interface qui spécifie une méthode **accept(visiteur)** représentée par un objet **Visiteur** qui doit lui être appliquée;

Patron VISITEUR

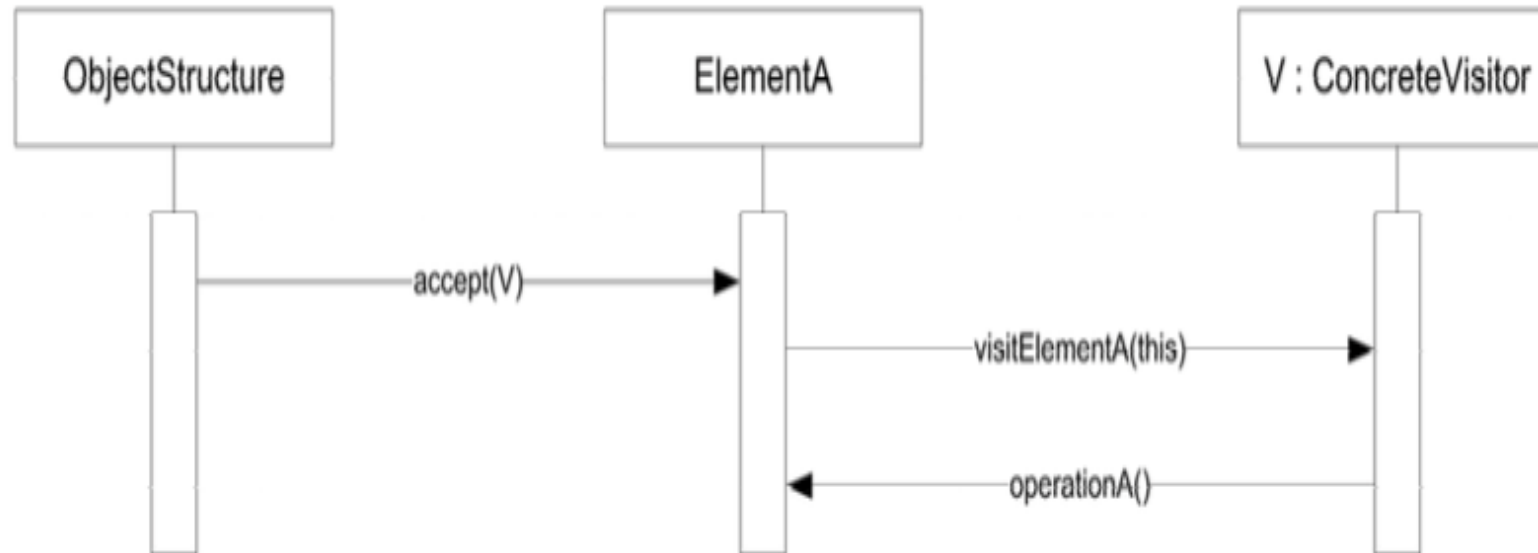


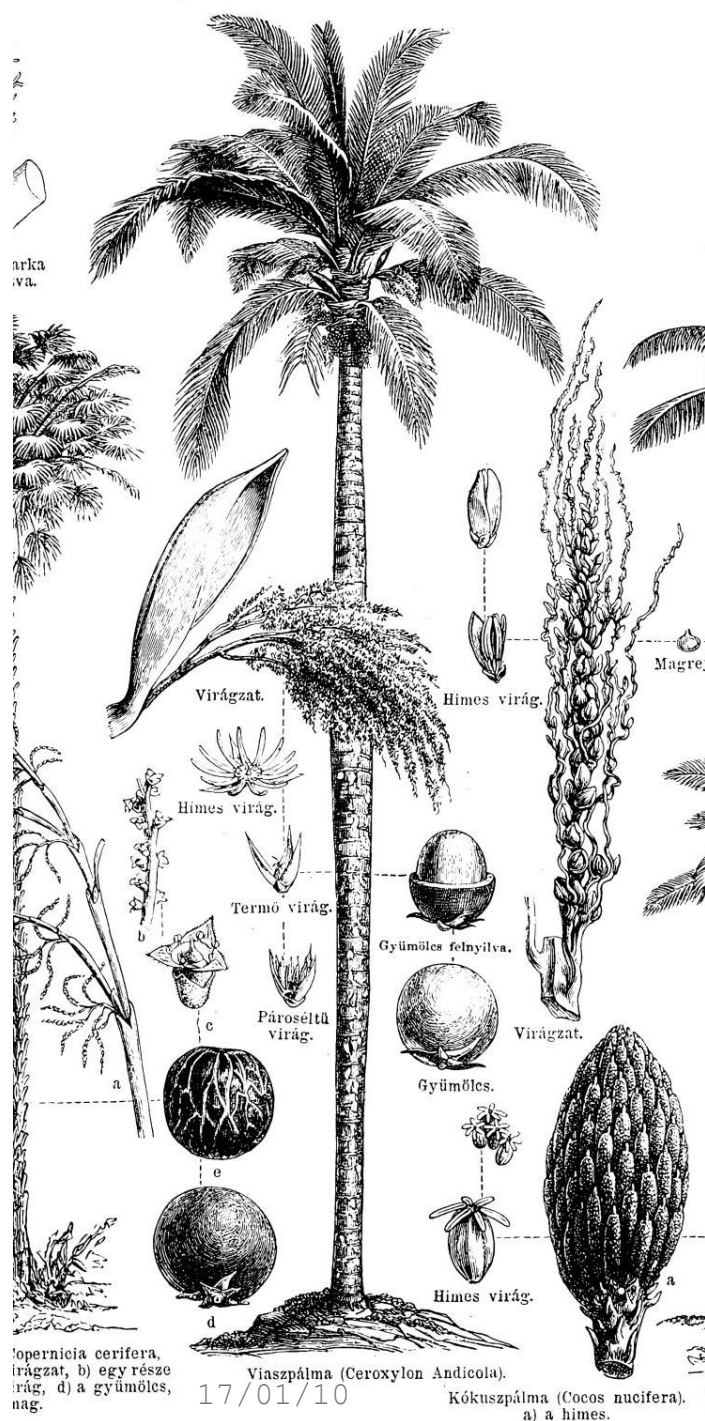
Solution

- Les classes de la structure doivent implanter cette interface;
 - Cette méthode fait appel à la méthode **visit()** du **Visiteur** passé en paramètre qui est spécifique au type de la classe;
 - Lors de cet appel à la méthode **visit()**, l'objet se passe lui-même en paramètre pour permettre au **Visiteur** d'effectuer des requêtes sur cet objet
- Une opération = Un Visiteur

Patron VISITEUR

Séquence





Patron VISITEUR

Séquence:

- La **structure d'objets** est créée;
- Un objet **Visiteur** implémentant une opération est créé;
- Pour appliquer l'opération à un objet de la structure, on fait appel à la méthode **accept()** de l'objet en lui passant le **Visiteur**;
- Dans la méthode **accept()**, l'objet fait appel à la méthode **visit()** du **Visiteur** qui est spécifique à son type;
- L'opération qu'implante le **Visiteur** est alors lancée;
 - Le Visiteur fait appel aux méthodes de l'objet afin d'exécuter l'opération
 -

Patron VISITEUR

Conséquence :

- Facilite l'ajout de nouvelles opérations;
- Permet de regrouper les détails d'implantation d'une opération applicable sur des objets hétérogènes;
- Permet de séparer une opération des données;
- Permet de parcourir des éléments hétérogènes d'une hiérarchie;
- Les Visiteurs peuvent accumuler des informations recueillies dans différents objets visités

Patron VISITEUR

Conséquences :

- L'ajout de nouvelles classes sur lesquelles les opérations doivent être effectuées est difficile:
 - Demande de modifier tous les **Visiteurs** qui implantent l'interface
- Peut compromettre l'encapsulation d'une classe:
 - Les objets doivent offrir une interface suffisante pour permettre aux Visiteurs d'effectuer leurs opérations



17/01/10



FIN